MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

Belawadi, SrirangapatnaTq, Mandya-571477

DEPARTMENT OF CSE (Artificial Intelligence)

Assignment – Project Report

2025-26

Subject Name: Cloud Computing

Subject Code: M23BCS505B

Semester: 5th

Project Title: Quantum Computing Concepts in the Cloud

Submitted by: Team No:

Sl. No.	Student Name	USN.		CO's Mapping				Total	Scaled to
			CO1	CO2	CO3	CO4	CO5		
1	Shreyas A	4MH23CA051							

Verified and approved by:

Faculty Name: Prof. M J Yogesh

Signature:

Date:

Project GitHub Repository:

Link: https://github.com/Shreyasamruth/cc

1. INTRODUCTION

1.1 Background

Quantum computing introduces qubits, superposition, entanglement and quantum gates — concepts that are not easy to visualize for newcomers. A lightweight web-based simulator accessible via a browser helps students and researchers experiment with circuits, see measurement outcomes, and visualize state vectors and histograms. This project builds an educational web app that simulates quantum circuits on a classical simulator (Qi skit Aer) and can be deployed to a cloud hosting service so users can access it anywhere.

1.2 Motivation

Bridging the gap between theory and hands-on practice for quantum computing learners. The app focuses on giving immediate feedback: construct a circuit in the browser, run on a simulator via Python backend, and visualize results—without needing expensive quantum hardware.

1.3 Objectives

- Provide an interactive web UI to create and run simple quantum circuits.
- Use Python + Qiskit on the backend to simulate circuits.
- Return state visualizations and measurement histograms to the frontend.
- Deployable to cloud (Heroku/Render/Railway/AWS/GCP).
- Include learn-by-example modules: single qubit superposition, Bell pair (entanglement), simple quantum teleportation demo (optional).

1.4 Scope

In-Scope: Circuit creation (preset circuits + simple manual gates), simulation with Qiskit Aer, result visualization (counts, histogram), cloud deployment instructions.

Out-of-Scope: Full drag-and-drop circuit designer, connection to real quantum hardware (but future extension possible), full multi-user persistence (simple DB optional).

2. LITERATURE REVIEW

2.1 Existing Systems

1. IBM Quantum Experience (IBM Quantum Composer & Lab)

IBM provides a widely used cloud-based quantum computing platform where users can:

- **Design quantum circuits visually**
- **Run them on simulators or real IBM quantum hardware**

- **Access Qiskit notebooks in the cloud**

  Limitations:
- **Requires IBM account**
- **Hardware access often limited via queue or tokens**
- **Complex for beginners to build custom web apps on top**

2. Google Cirq + Quantum Playground

Google Cirq is a Python library for quantum programming. The older "Quantum Playground" provided:

- **Browser-based visualization of quantum states**
- **Real-time circuit simulation**

  Limitations:
- **Playground is not actively maintained**
- **Cirq has no built-in cloud UI; users must script everything manually**

2.2 Key Concepts

Quantum computing is built on principles that differ fundamentally from classical computing. The simulation app developed in this project focuses on visualizing and explaining these foundational ideas. The key concepts relevant to this system are outlined below.

1. Qubits

A *quantum bit* (qubit) is the basic unit of quantum information.
Unlike classical bits, which are strictly **0** or **1**, a qubit can exist in a combination of both states simultaneously.

Mathematically:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

Where $\alpha$and $\beta$are probability amplitudes.

2. Superposition

Superposition allows a qubit to hold multiple states at once.
This means it behaves like **0 and 1 at the same time**, until it is measured.

In the app:

- Hadamard gate (H) is used to create superposition.
- Measurement collapses the state into classical values (0 or 1).

3. Entanglement

Entanglement is a uniquely quantum phenomenon where two or more qubits become correlated such that the state of one instantly influences the other, regardless of distance.

In the app:

- The Bell State demo uses **H** and **CX** gates to generate entangled qubits.
- Measured results show strong correlations (00 or 11).

4. Quantum Gates

Quantum gates manipulate qubits just like logic gates manipulate bits, but they are **reversible** and operate using linear algebra.

Common gates used in this system:

- **Hadamard (H):** Creates superposition
- **Pauli-X (X):** Quantum NOT
- **Pauli-Z (Z):** Phase shift
- **Controlled-NOT (CX):** Creates entanglement
- **Measurement:** Converts quantum state → classical output

These gates form circuits simulated by the backend.

5. Quantum Circuits

A quantum circuit is a sequence of gates applied to one or more qubits.
A typical circuit includes:

- Initialization
- One or more gates
- Measurement

The simulator converts user-selected demos (Hadamard, Bell pair) into circuits executed via Qiskit Aer.

6. Quantum Measurement

Measurement forces qubits to collapse into classical values.
The probabilities depend on the amplitudes of the state.

Example:

- A qubit in equal superposition should yield ~50% 0 and ~50% 1.
- Entangled qubits yield correlated outcomes.

7. Quantum Simulation

Running quantum circuits on classical hardware is known as **quantum simulation**.

In the project:

- The Flask backend uses **Qiskit Aer**, a high-performance simulator.
- Results are returned as JSON and displayed in the frontend.

Simulation enables learning quantum concepts without actual quantum hardware.

8. Cloud-Based Access

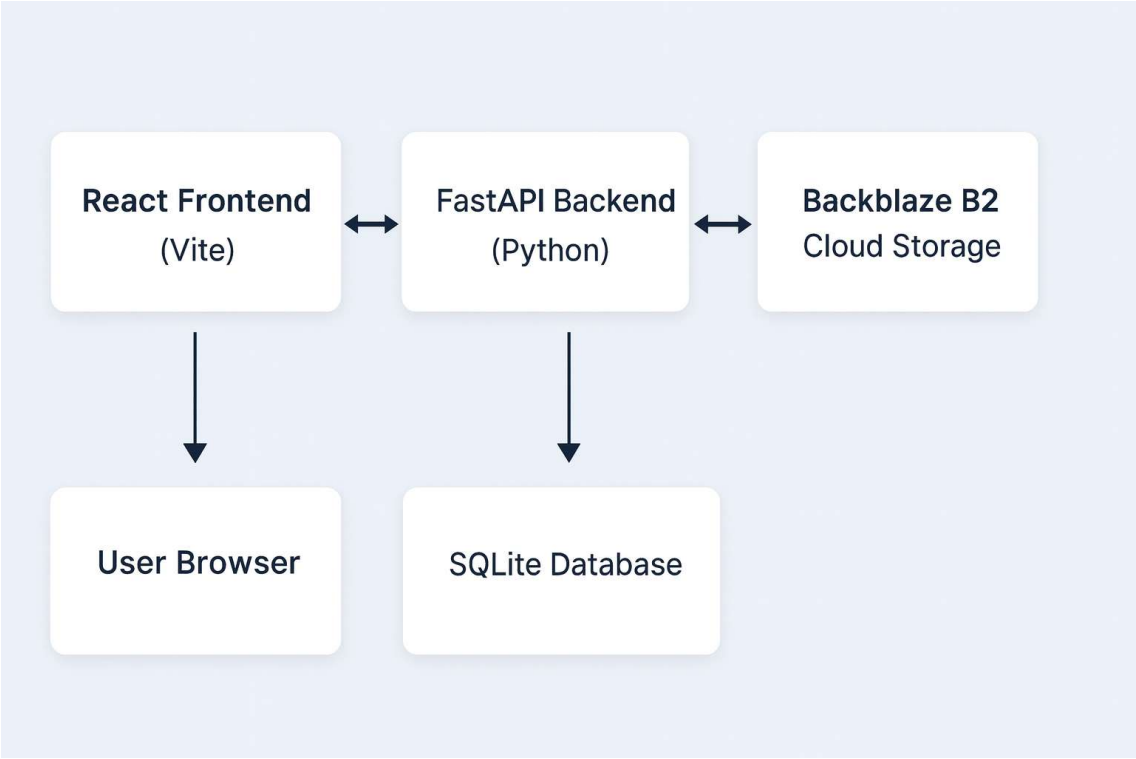
The system is designed to run on cloud platforms, enabling:

- Browser-based access from anywhere
- Lightweight interactions without installing Qiskit locally
- Instant visualization of quantum circuit behavior

Cloud deployment makes quantum learning scalable and accessible.

3. SYSTEM DESIGN

3.1 System Architecture Diagram



4. IMPLEMENTATION DETAILS

Step 1: Frontend Interface Creation (HTML, CSS, JS)

A simple user interface is developed using **HTML/CSS** for layout and **JavaScript** for interaction.

Users select a quantum demo (Hadamard, Bell pair), enter the number of shots, and click RUN to start the simulation.

Step 2: Sending Simulation Requests (Fetch API)

When the user clicks RUN, JavaScript sends a **POST** request to the Flask backend using the Fetch API.

The request contains:

- selected demo,
- number of shots,
- optional custom gate data.

Step 3: Flask Backend Processes Input

The Flask server receives the request and passes the data to the simulation module. The backend:

- builds the quantum circuit based on user input,
- validates demo/gate structure,
- prepares the circuit for simulation.

Step 4: Qiskit Aer Simulates the Quantum Circuit

The simulator.py module uses **Qiskit Aer** to run the circuit and generate:

- measurement counts,
- probability distribution results,
- histogram plot (PNG), which is later encoded as base64.

Step 5: Backend Sends Results to Frontend

The Flask backend returns a JSON response containing:

- measurement counts,
- base64-encoded histogram image.

The response is lightweight and efficient for cloud deployment.

Step 6: Frontend Displays Output

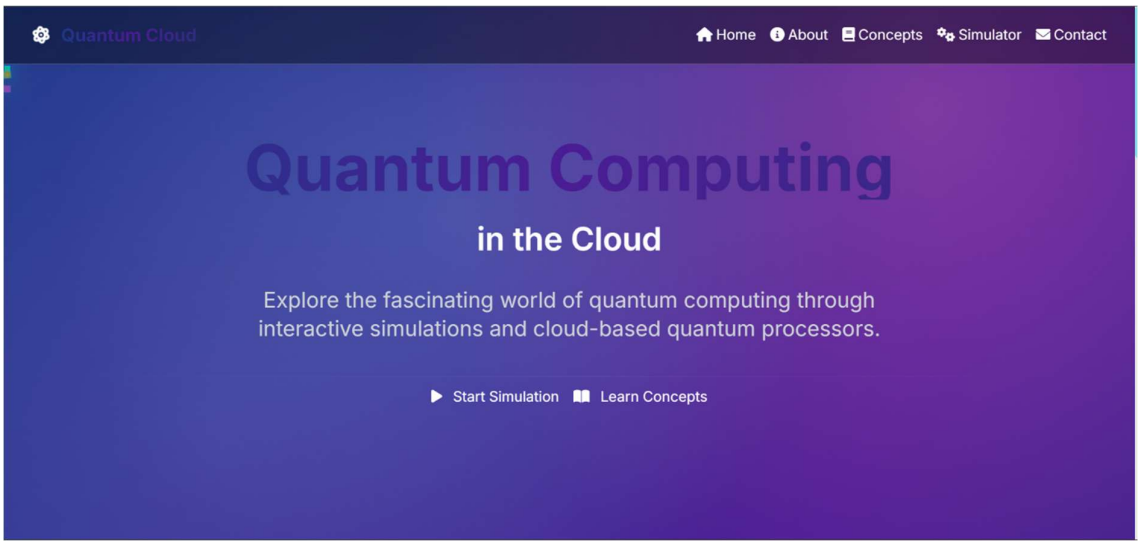
JavaScript receives the JSON, updates the page, and displays:

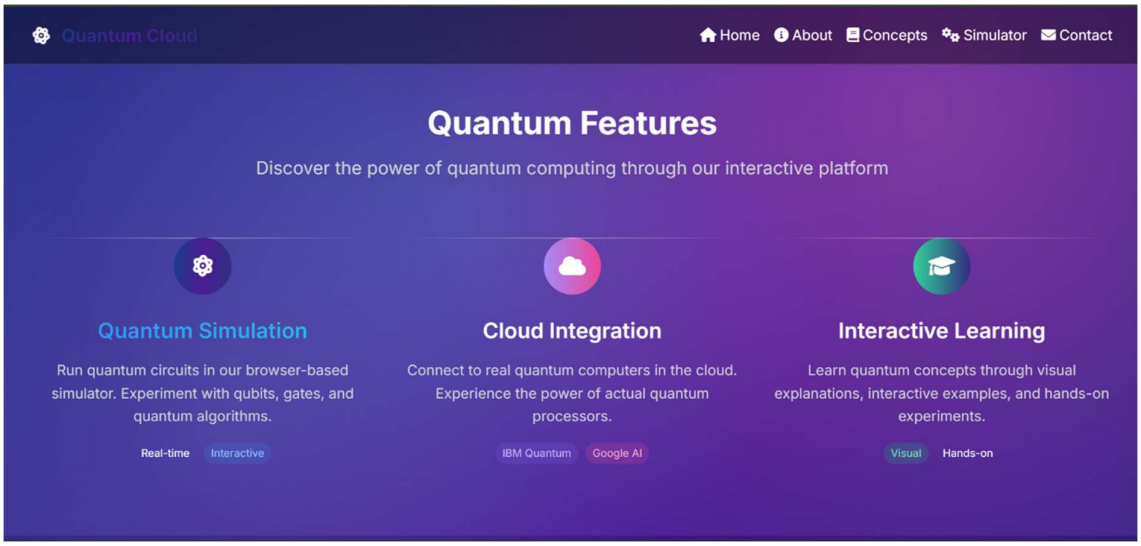
- the result counts,
- the histogram of measurements.

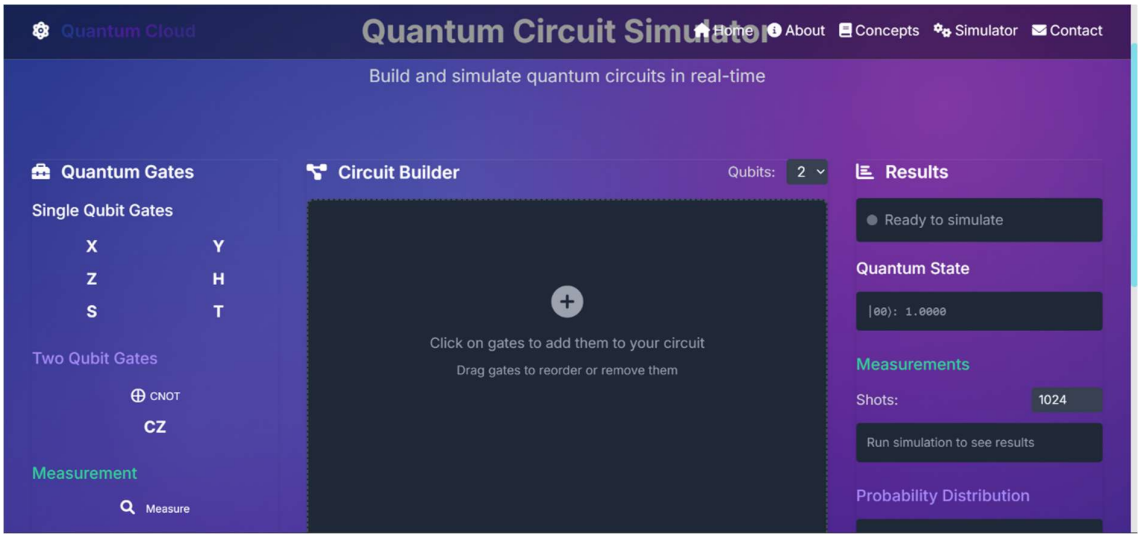
Users can instantly observe quantum behavior like superposition and entanglement.

5. RESULTS AND ANALYSIS

5.1 Output Screenshots







5.2 Advantages

1. Easy Accessibility

The system runs on any web browser, requiring no local installation of Qiskit or Python.

2. Cloud-Ready Design

Users can run simulations from anywhere, making it ideal for students and online courses.

3. Lightweight and Fast

Uses classical simulation (Qiskit Aer), which is quick and efficient for small circuits.

4. Educational Value

Clearly demonstrates fundamental quantum concepts like:

- **superposition**
- **entanglement**
- **measurement outcomes**

5. Simple and Intuitive UI

Minimalistic UI allows beginners to learn quantum computing without complexity.

5.3 Limitations

1. Limited to Small Circuits

Classical simulators struggle with circuits beyond 20–30 qubits due to exponential memory requirements.

2. No Real Quantum Hardware Integration

Currently only uses a simulator; does not run circuits on actual quantum devices.

3. Basic Visualization

Only histogram visualization is supported; no Bloch sphere or statevector animations.

4. Preset/Basic Circuits

Custom circuit building is limited; does not offer a full drag-and-drop quantum circuit editor.

5. No User Authentication

All users have the same access level; cannot save circuits or track user history.

5.4Future Enhancements

1. Integration with Real Quantum Hardware

Allow users to run circuits on IBM Quantum or Azure Quantum devices.

2. Advanced Visualizations

Add:

- **Bloch sphere animation**
- **Statevector visualization**
- **Quantum gate animations**

3. Drag-and-Drop Circuit Builder

A graphical interface for interactive circuit creation like IBM Composer or Quirk.

4. User Accounts & Saved Circuits

Enable users to:

- **create profiles**
- **save circuits**
- **share experiments**

5. Additional Quantum Algorithms

Add demo modules for:

- **Quantum Teleportation**
- **Grover's Algorithm**
- **Deutsch-Jozsa**
- **Quantum Fourier Transform**

6.1 Repository URL

https://github.com/Shreyasamruth/cc

7. CONCLUSION

The *Quantum Computing Concepts in the Cloud* system successfully demonstrates how complex quantum principles can be made accessible through a simple, browser-based simulation platform. By integrating Python, Flask, Qiskit Aer, and a lightweight HTML/CSS/JavaScript frontend, the application provides an interactive environment where users can experiment with fundamental quantum operations such as superposition, entanglement, and measurement.

The cloud-ready design ensures that users do not require powerful hardware or specialized installations to explore quantum circuits, making the system ideal for students, educators, and beginners entering the field of quantum computing. Although the current version focuses on essential concepts and small-scale simulations, it lays a strong foundation for future expansions such as real quantum hardware integration, advanced visualizations, and user-created circuits.

Overall, the project demonstrates that quantum computing education can be simplified, interactive, and accessible to anyone with an internet connection. This system represents a practical step toward bridging the gap between theoretical quantum concepts and hands-on experimentation.